

1. ASP.NET CORE & BACKEND

Q1. Middleware vs Filters vs Endpoint Filters — when should you use each?

Middleware runs in the HTTP pipeline for every request regardless of routing. Use it for cross-cutting concerns like logging, CORS, authentication, and exception handling.

Filters (Action, Resource, Authorization, Result, Exception) run within the MVC pipeline and have access to action context, model state, and result objects. Use filters for concerns tied to controller actions.

Endpoint Filters (introduced in .NET 7 minimal APIs) are lightweight, scoped to a specific endpoint or group. Use them for request validation or transformation on minimal API routes.

✓ **Key Point:** *Middleware = pipeline-wide; Filters = MVC-scoped; Endpoint Filters = minimal API scoped.*

Q2. How would you implement centralised exception handling?

Use a global exception-handling middleware (or `UseExceptionHandler`) as the first or second middleware in the pipeline so it wraps all subsequent middleware:

```
app.UseExceptionHandler(err => err.Run(async ctx => {
    var ex = ctx.Features.Get<IExceptionHandlerFeature>()?.Error;
    ctx.Response.StatusCode = 500;
    await ctx.Response.WriteAsJsonAsync(new { error = ex?.Message });
}));
```

In .NET 8+ you can implement `IExceptionHandler` and register it with `AddExceptionHandler()`. Combine with `ProblemDetails` for RFC 7807-compliant error responses.

✓ **Key Point:** *One place to handle all errors; return structured ProblemDetails.*

Q3. JWT vs Cookies Authentication — when should you use each?

JWT (Bearer tokens): Stateless, ideal for SPAs, mobile apps, and microservices. The token carries claims; no server-side session is needed. Downside: revocation requires a token blacklist or short expiry.

Cookie authentication: Stateful (session or encrypted cookie), automatic CSRF protection via `SameSite`, and simpler revocation by deleting the session. Best for traditional server-rendered apps.

- Use JWT for APIs consumed by JavaScript/mobile clients.
- Use cookies when the client is a browser-rendered MVC app.
- Hybrid: use refresh-token cookies + short-lived JWT access tokens for the best of both worlds.

Q4. How would you optimise async/await to improve scalability?

- Avoid `async void` — always return `Task/Task<T>` so exceptions propagate.
- Use `ConfigureAwait(false)` in library code to avoid deadlocks.
- Use `ValueTask` for hot paths that often complete synchronously.
- Avoid blocking calls (`.Result`, `.Wait()`) on async code.
- Use `IAsyncEnumerable` for streaming large result sets.
- Leverage cancellation tokens throughout the call chain.
- Use parallel async with `Task.WhenAll` for independent I/O operations.

Q5. How would you optimize API performance?

- Enable response caching and output caching (IOutputCache in .NET 7+).
 - Use compression middleware (Brotli/Gzip).
 - Minimise payload: use DTO projections, avoid over-fetching.
 - Implement pagination for collections.
 - Use asynchronous I/O throughout.
 - Profile with Application Insights or MiniProfiler to find bottlenecks.
 - Cache hot data in IMemoryCache or IDistributedCache (Redis).
 - Use HTTP/2 or HTTP/3 where possible.
-

Q6. What causes memory leaks in .NET applications?

- Forgetting to dispose IDisposable objects (streams, DbContext, HttpClient).
- Static references holding large object graphs.
- Event handler subscriptions not unsubscribed (+=, never -=).
- Incorrect use of MemoryCache without size limits.
- Long-lived closures capturing large variables.
- Finalizer queue buildup when Dispose is never called.

Use dotnet-counters, dotnet-dump, or the VS Diagnostic Tools to identify GC pressure and large object heap growth.

Q7. How would you design scalable APIs in ASP.NET Core?

- Follow REST principles; version APIs (/api/v1/...).
 - Stateless services — store session in distributed cache.
 - CQRS with MediatR to separate reads and writes.
 - Message queues (Service Bus, RabbitMQ) for async processing.
 - Horizontal scaling behind a load balancer.
 - Database read replicas for query-heavy workloads.
 - Rate limiting (RateLimiter in .NET 7+) to protect endpoints.
-

Q8. What is Dependency Injection in ASP.NET Core?

DI is built-in to ASP.NET Core via IServiceCollection. Services are registered with three lifetimes:

- Transient — new instance per injection.
- Scoped — one instance per HTTP request.
- Singleton — one instance for the application lifetime.

Use constructor injection (preferred) or IServiceProvider.GetService() for manual resolution. Avoid the Service Locator anti-pattern.

Q9. What is IHostedService and when to use it?

IHostedService allows background tasks to run within the ASP.NET Core host lifecycle. BackgroundService is the base class that simplifies implementation via ExecuteAsync(CancellationToken).

- Use for periodic jobs (polling queues, cleanup tasks).
 - Use for startup/shutdown hooks.
 - Use Hangfire or Azure Functions for more complex scheduling.
-

Q10. How does Caching improve performance?

- In-memory cache (IMemoryCache): sub-millisecond latency, single-node.
- Distributed cache (IDistributedCache / Redis): shared across pods.
- Response caching: cache entire HTTP responses.
- Output caching (.NET 7+): fine-grained cache policies per endpoint.
- CDN caching: cache static assets at the edge.

Always set appropriate expiry (absolute or sliding) and eviction policies to avoid stale data.

2. ANGULAR / REACT FRONTEND

Q1. Angular vs React — when should you choose each?

Angular: Full opinionated framework (DI, router, forms, HTTP client, RxJS). Best for large enterprise teams needing strong conventions, TypeScript-first, and a complete ecosystem.

React: UI library — you compose your own stack (React Query, Zustand, React Router). More flexible, larger community, faster iteration, and better for component libraries.

✓ **Key Point:** *Angular = batteries-included enterprise; React = flexible composable ecosystem.*

Q2. What causes unnecessary re-renders in React?

- Inline object/array/function literals in JSX create new references on every render.
- Parent component re-renders causing child re-renders.
- Context value changes triggering all consumers.
- Missing or wrong keys in lists.

Fixes: React.memo, useMemo, useCallback, and splitting context into smaller pieces.

Q3. How would you optimize frontend performance?

- Lazy loading routes and heavy components (React.lazy / Angular loadChildren).
- Code splitting and tree shaking via Webpack/Vite.
- Memoization: React.memo, useMemo, useCallback.
- Virtual scrolling for large lists (CDK VirtualScrollViewPort, react-window).
- Image optimisation: WebP, lazy loading, srcset.
- CDN for static assets, HTTP/2 multiplexing.
- Avoid layout thrashing; batch DOM reads/writes.

Q4. Lazy Loading — why is it important?

Lazy loading defers downloading code until it is actually needed, reducing the initial bundle size and Time-to-Interactive. In Angular, use loadChildren in the router config. In React, use React.lazy + Suspense.

✓ **Key Point:** *Smaller initial bundle → faster first paint → better UX and Core Web Vitals scores.*

Q5. State Management — when is it needed?

Local component state (useState/signals) suffices for UI-only state. Reach for global state management when:

- Multiple unrelated components need the same data.
- Data must persist across routes.
- Complex async flows (NgRx Effects, Redux Saga/Thunk).
- Undo/redo or time-travel debugging is needed.

Options: NgRx (Angular), Redux Toolkit, Zustand, Jotai, Recoil (React).

Q6. How would you secure frontend API communication?

- Store tokens in memory (not localStorage) to mitigate XSS.
 - Use HttpOnly cookies for refresh tokens.
 - Always communicate over HTTPS.
 - Implement CSRF tokens for cookie-based auth.
 - Validate and sanitise all inputs on both client and server.
 - Set strict Content-Security-Policy headers.
-

Q7. Reactive Forms vs Template-Driven Forms (Angular)?

Template-Driven: Simple, two-way binding with ngModel. Good for small forms.

Reactive Forms: Model-driven, defined in TypeScript. Better for complex validation, dynamic fields, unit testing, and large forms. Supports FormArray and custom validators.

✓ **Key Point:** *Prefer Reactive Forms for anything beyond a simple login form.*

Q8. How would you structure large frontend applications?

- Feature-based folder structure (each feature is self-contained).
 - Shared/core modules for common services and components.
 - Design system / component library for UI consistency.
 - Lazy-loaded feature modules / routes.
 - Clear separation: smart (container) vs dumb (presentational) components.
 - Use a monorepo (Nx, Turborepo) for multi-app projects.
-

Q9. Difference between Observables and Promises?

Promise: Single future value, eager (executes immediately), not cancellable.

Observable (RxJS): Stream of zero or more values over time, lazy (executes on subscribe), cancellable (unsubscribe), composable with operators (map, filter, switchMap, etc.).

✓ **Key Point:** *Use Observables for HTTP streams, WebSockets, and event streams. Promises for one-shot async ops.*

Q10. How would you handle route guards / protected routes?

Angular: Implement CanActivate/CanMatch guards; check auth state and redirect to login if not authenticated.

React: Create a PrivateRoute wrapper component that checks auth state and redirects with Navigate.

Always validate authorization on the server — client-side guards are UX only, not security.

3. SQL SERVER & DATABASE

Q1. Clustered vs Non-Clustered Index — which to use?

Clustered Index: Physically sorts the table rows. Only one per table. Best on the primary key or the most common range-query column.

Non-Clustered Index: Separate structure with pointers to the row. Many per table. Best on frequently filtered/joined columns.

✓ **Key Point:** *Clustered = data order; Non-Clustered = lookup pointers. Use covering indexes to avoid key lookups.*

Q2. How would you optimise slow queries?

- Run SET STATISTICS IO, TIME ON and examine the execution plan.
 - Add or tune indexes (avoid index scans; prefer seeks).
 - Rewrite correlated subqueries as JOINS or CTEs.
 - Avoid SELECT *; project only needed columns.
 - Use query hints sparingly (NOLOCK, RECOMPILE).
 - Partition large tables.
 - Update statistics and rebuild fragmented indexes.
 - Cache results at application layer for static data.
-

Q3. What causes deadlocks?

Deadlocks occur when two (or more) transactions hold locks the other needs, forming a circular wait. SQL Server's deadlock monitor kills one transaction (victim) and raises error 1205.

- Access tables in the same order across transactions.
 - Keep transactions short.
 - Use READ COMMITTED SNAPSHOT isolation to reduce lock contention.
 - Use appropriate index coverage to reduce lock duration.
-

Q4. CTE vs Temp Table?

CTE (Common Table Expression): Named sub-query defined with WITH. In-line, no physical storage, evaluated each time it is referenced. Good for readability and recursive queries.

Temp Table (#table): Physically stored in tempdb. Allows indexes, statistics, and multiple references without re-evaluation. Better for large datasets referenced multiple times.

✓ **Key Point:** CTE = readability / recursion; Temp Table = reuse large intermediate result sets.

Q5. How do execution plans help optimization?

Execution plans show the operations SQL Server performs to satisfy a query. Key things to look for:

- Table Scan vs Index Seek — seek is almost always preferable.
 - Key Lookup — consider a covering index.
 - Sort operator on large datasets — add an ordered index.
 - Estimated vs actual rows — large difference means stale statistics.
 - Warnings (missing index, implicit conversion).
-

Q6. How would you handle millions of records efficiently?

- Pagination: OFFSET/FETCH or keyset (seek) pagination.
 - Partitioning: partition table by date or region.
 - Archiving: move old data to archive tables.
 - Read replicas or CQRS read store.
 - Bulk insert with SqlBulkCopy or BULK INSERT.
 - Columnstore indexes for analytical workloads.
 - Caching aggregates in Redis.
-

Q7. What is parameter sniffing?

SQL Server caches execution plans based on the first set of parameter values. If those values are atypical (e.g., very small or very large), the cached plan may be suboptimal for subsequent calls.

Solutions: OPTION (RECOMPILE), OPTIMIZE FOR, local variables to break sniffing, or updating statistics.

Q8. How do you design scalable DB architecture?

- Normalise (3NF) for OLTP; denormalise for OLAP/reporting.
 - Separate read and write models (CQRS + read replicas).
 - Use connection pooling.
 - Sharding for extreme scale.
 - Event sourcing for audit and replay.
 - Database-per-service in microservices.
-

Q9. Stored Procedure vs Function — differences?

Stored Procedure: Can modify data (INSERT/UPDATE/DELETE), use transactions, return multiple result sets, and use output parameters. Cannot be used inside SELECT.

Function (Scalar/TVF): Must return a value, no side effects (mostly), can be used inside SELECT. Scalar UDFs can inhibit parallelism — prefer inline TVFs.

✓ **Key Point:** Use stored procs for complex data-modification logic; inline TVFs for reusable query fragments.

4. CLOUD, DEVOPS & ARCHITECTURE

Q1. Azure App Service vs AKS — when to use each?

App Service (PaaS): Managed hosting for web apps and APIs. No infrastructure management. Best for standard web apps, REST APIs, and small-to-medium teams.

AKS (Kubernetes): Full container orchestration. Best for microservices, complex scaling policies, multi-container apps, and teams comfortable with Kubernetes.

✓ **Key Point:** App Service = simplicity and speed; AKS = control and scale.

Q2. Docker vs Kubernetes — when should you use each?

Docker: Containerises a single application. Use Docker (and Docker Compose) for local development and simple single-host deployments.

Kubernetes: Orchestrates many containers across a cluster — scheduling, scaling, self-healing, service discovery. Use when you need production-grade container management at scale.

Q3. How would you implement CI/CD pipelines?

- Source control trigger (push/PR) starts the pipeline.
 - Build stage: restore, compile, run unit tests.
 - Quality gates: SonarQube / code coverage thresholds.
 - Containerise: Docker build + push to Azure Container Registry.
 - Deploy to staging: Helm upgrade / App Service slot swap.
 - Integration/smoke tests against staging.
 - Approval gate → deploy to production (blue-green or canary).
 - Tools: Azure DevOps Pipelines, GitHub Actions.
-

Q4. Monolith vs Microservices — trade-offs?

Monolith: Simple to develop, test, and deploy initially. Risk: big-ball-of-mud as it grows; difficult to scale individual components; long release cycles.

Microservices: Independent deployability, technology heterogeneity, fault isolation, fine-grained scaling. Risk: distributed systems complexity (network, latency, data consistency).

Q5. How would you secure secrets in production?

- Never store secrets in code or config files.
 - Use Azure Key Vault; reference via Managed Identity (no credentials in code).
 - Rotate secrets regularly; use Key Vault versioning.
 - Use GitHub/Azure DevOps secret variables in pipelines.
 - Audit secret access with Key Vault diagnostic logs.
-

Q6. How would you design zero-downtime deployment?

- Blue-Green: run two identical environments; switch traffic instantly.
 - Canary: gradually shift a small % of traffic to the new version.
 - Rolling: replace instances one at a time.
 - Feature flags: deploy code but enable features selectively.
 - Database: use backward-compatible migrations (add columns, never drop).
-

Q7. How would you monitor distributed applications?

- Structured logging with Serilog → Application Insights / ELK.
 - Distributed tracing: Azure Application Insights, OpenTelemetry.
 - Metrics dashboards: Azure Monitor, Grafana.
 - Alerting: alert on error rate, p99 latency, pod restarts.
 - Health checks endpoint (/health) consumed by load balancer.
-

Q8. How would you handle service-to-service communication?

Synchronous: REST (HttpClient + Polly for retry/circuit-breaker) or gRPC for performance-critical paths.

Asynchronous: Message broker (Azure Service Bus, RabbitMQ) for decoupled, reliable messaging.

✓ **Key Point:** *Prefer async messaging for resilience; use gRPC for high-throughput internal APIs.*

Q9. What are cascading failures and how to prevent them?

A cascading failure occurs when the failure of one service causes dependent services to fail, propagating through the system.

- Circuit Breaker (Polly): stop calling a failing service after N failures; allow recovery.
 - Bulkhead: limit concurrent calls to a dependency.
 - Timeout: never wait indefinitely.
 - Fallback: return cached or default data.
 - Retry with exponential backoff and jitter.
-

Q10. How would you ensure high availability and disaster recovery?

- Multi-region Active-Active or Active-Passive deployment.
 - Azure Traffic Manager / Front Door for geographic routing.
 - Define RTO (Recovery Time Objective) and RPO (Recovery Point Objective).
 - Automated failover with Azure Site Recovery.
 - Regular backup testing and DR drills.
 - Health checks and auto-scaling to handle traffic spikes.
-

5. REAL-TIME SCENARIO QUESTIONS

Q1. API response time suddenly increased in production. What would you do?

- Check Application Insights / logs for error spikes and slow dependencies.
 - Analyse slow endpoints: identify which routes regressed.
 - Check DB: look for new slow queries, missing indexes, or lock waits.
 - Check DB queries & indexes — run `sp_BlitzCache`.
 - Review recent deployments — correlate timing with releases.
 - Check external dependencies (third-party APIs, queues).
 - Identify bottlenecks (CPU, memory, thread pool exhaustion) and fix.
-

Q2. Application traffic increased 10x overnight. What would you do?

- Enable Auto Scaling (App Service scale-out rules or AKS HPA).
 - Use a Load Balancer to distribute traffic.
 - Optimise DB: read replicas, connection pool tuning.
 - Enable CDN for static assets.
 - Review unnecessary re-renders (frontend).
 - Optimise API performance: caching, compression.
 - Monitor & set alerts for CPU, memory, and response time.
-

Q3. Frontend became slow after deployment. What would you do?

- Check Network tab in DevTools: identify large/slow requests.
 - Analyse bundle size (webpack-bundle-analyzer).
 - Check lazy loading is working correctly.
 - Review unnecessary re-renders with React DevTools.
 - Optimise images & assets (WebP, lazy loading).
 - Profile performance with Lighthouse / Chrome Performance tab.
-

Q4. One SQL query checks multiple transactions — it is slow. What would you do?

- Check blocking sessions (DMVs: `sys.dm_exec_requests`).
 - Analyse execution plan: look for scans, key lookups.
 - Identify missing indexes with `sys.dm_db_missing_index_details`.
 - Optimise query / add indexes (covering index).
 - Avoid long transactions; commit early.
 - Use appropriate isolation level (READ COMMITTED SNAPSHOT).
-

Q5. One microservice keeps failing repeatedly. What would you do?

- Check logs & error details (Application Insights, Serilog).
 - Investigate root cause: unhandled exception, resource exhaustion.
 - Implement retries & circuit breaker (Polly).
 - Use health checks (/health endpoint).
 - Isolate failing service with bulkhead pattern.
 - Prevent cascading failures to upstream services.
-

Q6. Need to deploy without downtime. What would you do?

- Use Blue-Green Deployment: deploy new version alongside old, switch traffic.
- Use rolling updates (Kubernetes rolling strategy).
- Health checks & monitoring before full cutover.
- Database backward compatibility (additive migrations only).
- Automated rollback strategy if health checks fail.
- Feature flags to decouple deployment from release.

6. DESIGN & ARCHITECTURE

Q1. How would you design a scalable e-commerce application?

- Microservices: Product, Order, Payment, Inventory, User services.
- API Gateway (Azure API Management) as single entry point.
- Event-driven: Service Bus for order events (OrderPlaced, StockReserved).
- CQRS: separate read models (Elasticsearch for product search).
- Redis caching for product catalogue and sessions.
- CDN for images and static assets.
- Saga pattern for distributed transactions (e.g., order fulfilment).

Q2. How would you separate concerns in a large application?

Use Clean Architecture (Onion/Hexagonal):

- Domain layer: entities, value objects, domain events — no external dependencies.
- Application layer: use cases, commands, queries (MediatR).
- Infrastructure layer: EF Core, external APIs, file storage.
- Presentation layer: controllers, minimal APIs, SignalR hubs.
- Dependencies point inward; domain knows nothing about infrastructure.

Q3. What is Clean Architecture?

Clean Architecture (Robert C. Martin) organises code in concentric layers where dependencies point inward. The innermost layer (Domain) has no dependencies; outer layers depend on inner ones but not vice versa. This makes the domain logic testable and independent of frameworks, databases, and UI.

Q4. What is CQRS and Traditional Architecture?

Traditional: One model handles reads and writes. Simple but becomes a bottleneck at scale.

CQRS (Command Query Responsibility Segregation): Separate models for writes (commands) and reads (queries). Commands mutate state; queries return data from an optimised read store. Enables independent scaling and optimisation of each side.

✓ **Key Point: Combine CQRS with Event Sourcing for full audit trail and temporal queries.**

Q5. When would you use Event-Driven Architecture?

- Decoupling services that should not call each other synchronously.
- Reliable async processing (order fulfilment, email notifications).
- Fan-out: one event triggers multiple consumers.
- Audit log / event sourcing.
- Long-running workflows (Azure Durable Functions, Saga).

Q6. How would you handle distributed transactions?

Two-Phase Commit (2PC): Strong consistency but high coupling and risk of blocking.

Saga Pattern (preferred for microservices):

- Choreography: each service listens for events and reacts (decoupled but harder to trace).
- Orchestration: a central orchestrator (Durable Functions) directs steps and handles compensation.

✓ **Key Point:** *Avoid distributed transactions where possible; design for eventual consistency.*

7. SECURITY

Q1. How would you secure APIs?

- Authentication: JWT Bearer tokens or OAuth 2.0 / OpenID Connect.
- Authorization: role-based (RBAC) and policy-based (claims).
- Always use HTTPS; enforce HSTS.
- Rate limiting to prevent brute force and DoS.
- Input validation and output encoding.
- CORS: whitelist only trusted origins.
- API versioning to avoid breaking changes.

Q2. How does OAuth 2.0 / OpenID Connect work?

OAuth 2.0 is an authorisation framework. The client obtains an access token from an authorisation server (e.g., Azure AD) and presents it to the resource server (your API).

OpenID Connect adds an identity layer on top of OAuth 2.0, providing an ID token (JWT) with user claims. Use PKCE flow for SPAs and mobile apps.

Q3. What is CORS and why is it important?

Cross-Origin Resource Sharing (CORS) is a browser security mechanism that restricts web pages from making requests to a different domain. Configure CORS in ASP.NET Core with `AddCors()` and `UseCors()`. Whitelist only trusted origins in production — never use `AllowAnyOrigin` with `AllowCredentials`.

Q4. How would you prevent SQL Injection?

- Always use parameterised queries or ORM (EF Core parameterises by default).
- Never concatenate user input into SQL strings.
- Validate and sanitise inputs.
- Principle of least privilege for DB user accounts.
- Use stored procedures with parameters.
- Enable Web Application Firewall (WAF).

Q5. How would you secure secrets and configuration?

- Azure Key Vault: store connection strings, API keys, certificates.
- Managed Identity: services authenticate to Key Vault without any credentials in code.
- App Configuration: feature flags and non-sensitive config.
- Never commit secrets to source control (`.gitignore` `appsettings.Production.json`).
- Use GitHub/Azure DevOps secret variables in CI/CD pipelines.

Q6. How would you implement role-based access control?

In ASP.NET Core, add `[Authorize(Roles = "Admin")]` to controllers/actions. Define roles as claims in the JWT. For fine-grained control, use Policy-Based Authorization:

```
builder.Services.AddAuthorization(opts =>
    opts.AddPolicy("CanEditOrders", p => p.RequireClaim("permission", "orders.edit"));
```

Manage roles and permissions in Azure AD App Roles or a custom roles table.

Q7. What security headers should be used?

- Content-Security-Policy (CSP): restricts script sources to prevent XSS.
- X-Content-Type-Options: nosniff
- X-Frame-Options: DENY (clickjacking prevention)
- Strict-Transport-Security (HSTS): enforce HTTPS.
- Referrer-Policy: limit referrer information.
- Permissions-Policy: restrict browser feature access.

8. TESTING

Q1. Unit Test vs Integration Test?

Unit Test: Tests a single class/method in isolation with all dependencies mocked. Fast, no I/O.

Integration Test: Tests multiple components together (e.g., controller → service → DB). Uses `WebApplicationFactory` in ASP.NET Core. Slower but catches integration bugs.

✓ **Key Point:** *Unit tests = logic correctness; Integration tests = component interaction.*

Q2. How would you write unit tests for business logic?

- Arrange-Act-Assert (AAA) pattern.
- Use xUnit or NUnit as test framework.
- Mock dependencies with Moq or NSubstitute.
- Test happy path, edge cases, and exception paths.
- Keep tests independent and deterministic.
- Name tests clearly: `MethodName_Scenario_ExpectedResult`.

Q3. What is mocking and why is it used?

Mocking replaces real dependencies with controllable fakes so tests run fast and in isolation. You can control return values, verify method calls, and simulate exceptions without hitting real databases or APIs.

```
var mockRepo = new Mock<IOrderRepository>();
mockRepo.Setup(r => r.GetByIdAsync(1)).ReturnsAsync(new Order { Id = 1 });
```

Q4. How would you test Web APIs?

- Unit test: test controller logic with mocked services.
- Integration test: use `WebApplicationFactory` + `HttpClient` to test full pipeline.
- Contract testing: Pact for consumer-driven contracts in microservices.
- Load testing: k6, JMeter, or Azure Load Testing.
- Use `TestServer` for in-memory API testing without network overhead.

Q5. How would you do frontend testing?

- Unit: Jest + React Testing Library for component logic.
 - Component: Storybook for visual isolation.
 - Integration: Testing Library user-event for user interactions.
 - E2E: Playwright or Cypress for full browser automation.
 - Snapshot testing for UI regression detection.
-

Q6. What tools do you use for testing?

- Backend: xUnit, Moq, FluentAssertions, Bogus (test data), AutoFixture.
 - Frontend: Jest, React Testing Library, Playwright, Cypress.
 - API: Postman, REST Client (VS Code).
 - Load: k6, Azure Load Testing.
 - Coverage: Coverlet + ReportGenerator.
-

Q7. How would you achieve high test coverage?

- Test all business logic branches (if/else, switch cases).
 - Test exception paths and edge cases.
 - Use code coverage reports to identify gaps (Coverlet).
 - Enforce minimum coverage thresholds in CI pipeline.
 - Focus on meaningful coverage over 100% line coverage.
 - Mutation testing (Stryker.NET) to ensure tests are actually effective.
-

9. OOPs & .NET CONCEPTS

Q1. Abstract vs Interface?

Abstract Class: Can have implementation, fields, constructors. Supports single inheritance.

Interface: Contract only (though C# 8+ allows default implementations). Supports multiple implementation.

✓ **Key Point:** Use interfaces for cross-cutting contracts; abstract classes for shared base behaviour.

Q2. Inheritance vs Composition?

Inheritance (is-a): Subclass inherits behaviour. Tight coupling; fragile base-class problem.

Composition (has-a): Class delegates to composed objects. More flexible and testable.

✓ **Key Point:** Prefer composition over inheritance (GoF principle). Use inheritance only for true is-a relationships.

Q3. What is Dependency Injection?

DI is a design pattern where an object receives its dependencies from an external source rather than creating them itself. It promotes loose coupling, testability, and adherence to SOLID principles. In ASP.NET Core, the built-in IoC container manages object lifetimes.

Q4. IEnumerable vs IQueryable?

IEnumerable<T>: In-memory query execution. LINQ operators run in C#. Data already fetched from DB.

IQueryable<T>: Query is translated to SQL and executed on the database. Only returns needed data.

✓ **Key Point:** Always use IQueryable with EF Core when filtering large datasets; use IEnumerable for in-memory operations.

Q5. Value Types vs Reference Types?

Value Types (struct, int, bool, DateTime): Stored on the stack, copied on assignment.

Reference Types (class, string, arrays): Stored on the heap, assignment copies the reference.

Key implication: mutating a value type copy does not affect the original; mutating a reference type copy does.

Q6. Garbage Collection in .NET?

The CLR's GC automatically manages memory. Objects are allocated on the managed heap and collected when unreachable. GC has three generations (0, 1, 2); Gen 0 is collected most frequently (short-lived objects). The Large Object Heap (LOH, >85 KB) is collected less frequently.

Best practices: Implement IDisposable for unmanaged resources; use using statements; avoid finalizers.

Q7. Difference between Task and Thread?

Thread: OS-level; heavyweight (~1 MB stack); explicit management.

Task: Abstraction over the ThreadPool; lightweight; supports async/await, cancellation, and chaining.

✓ **Key Point:** Always prefer Task/async-await over raw Thread for I/O-bound work.

10. SOFT SKILLS (IMPORTANT!)

Q1. Tell me about yourself.

Structure with the Present-Past-Future formula: current role and key achievements → background that led here → why you are excited about this opportunity at Accenture. Keep it to 2 minutes, technical and confident.

Q2. Why do you want to join Accenture?

- Global scale: opportunity to work on enterprise-grade projects across industries.
 - Learning culture: Accenture's continuous learning platforms and certifications.
 - Innovation: exposure to cutting-edge Azure, AI, and cloud-native technologies.
 - Impact: delivering solutions that matter to large organisations and millions of users.
-

Q3. What is your biggest strength and weakness?

Strength: Choose one that is genuinely relevant — e.g., 'I systematically break down complex problems into smaller deliverables, which helps me deliver quality code on time.'

Weakness: Choose a real one with a growth story — e.g., 'I used to spend too long perfecting code before shipping. I now timebox tasks and rely on iterative feedback to strike the right balance.'

Q4. How do you handle pressure?

- Prioritise and break work into smaller milestones.
 - Communicate blockers early to stakeholders.
 - Focus on what is within your control.
 - Use retrospectives to learn and prevent recurrence.
 - Maintain energy through structured work hours and breaks.
-

Q5. How do you stay updated?

- Microsoft Learn, Pluralsight, and official .NET blog.
 - Follow community: Scott Hanselman, Nick Chapsas, Milan Jovanović.
 - Attend NDC, DevConf, and local .NET meetups.
 - Build side projects to apply new concepts.
 - Read architecture ADRs and RFC proposals.
-

Q6. Describe a challenging situation you solved.

Use the STAR method: Situation → Task → Action → Result. Pick a real technical challenge (e.g., performance issue, production incident, tight deadline). Emphasise your specific contribution, the reasoning behind your decisions, and the measurable outcome.

Q7. How do you work in a team?

- Communicate clearly and proactively.
 - Pair program and code-review constructively.
 - Share knowledge via documentation and demos.
 - Respect different opinions; aim for technical consensus.
 - Celebrate team wins and take collective ownership of failures.
-

11. COMMON TOOLS & TECHNOLOGIES

Category	Primary Tools / Technologies
Backend	ASP.NET Core, Web API, gRPC, MediatR, AutoMapper
Frontend	Angular, React, TypeScript, HTML5, CSS3, Bootstrap, Tailwind
Caching	Redis, IMemoryCache, IDistributedCache, Output Cache
Cloud	Azure App Service, AKS, Functions, Service Bus, Storage, Key Vault
DevOps	Azure DevOps, GitHub Actions, Docker, Helm, Terraform
Containers	Docker, Kubernetes, Azure Container Registry
Monitoring	Application Insights, Azure Monitor, Serilog, OpenTelemetry
Messaging	Service Bus, RabbitMQ, Azure Event Grid
Testing	xUnit, Moq, FluentAssertions, Playwright, Jest
ORM	Entity Framework Core, Dapper
Auth	Azure AD / Entra ID, OAuth 2.0, OpenID Connect, JWT

12. INTERVIEW DO'S & DON'TS

DO'S	DON'TS
✓ Understand requirements clearly before coding. ✓ Write clean & maintainable code. ✓ Think of scalability & security in every design. ✓ Discuss performance implications proactively. ✓ Write unit tests — mention TDD even if you practise it partially. ✓ Communicate blockers early and transparently. ✓ Take ownership of your work and decisions.	✗ Don't over-engineer simple solutions. ✗ Don't hardcode values — use configuration. ✗ Don't ignore error handling or edge cases. ✗ Don't write unparameterised queries (SQL injection risk). ✗ Don't commit secrets to source control. ✗ Don't blame others for team failures. ✗ Don't stop learning — mention your learning habits.

13. INTERVIEW PREPARATION CHECKLIST

- Revise .NET Core, OOPs, Angular/React fundamentals.
- Practice DSA & problem solving (LeetCode Medium level).
- Work on real-world projects you can discuss in depth.
- Prepare for scenario-based questions (STAR method).
- Review Azure, DevOps & CI/CD concepts.
- Improve communication skills — practice out loud.
- Mock interviews & feedback (peers or platforms like Pramp).

- Keep learning and stay updated (latest .NET, Azure announcements).
- Prepare 3-5 impactful stories from your experience.
- Research Accenture's recent projects, values, and culture.

PREPARE SMART. SOLVE BETTER. BUILD IMPACT. GROW WITH ACCENTURE.